

Lab: Reflected DOM XSS

1. Mục đích của bài lab

→ Khai thác lỗ hổng DOM-based Cross-Site Scripting do ứng dụng sử dụng hàm eval() để xử lý dữ liệu JSON và không escape ký tự backslash (\) đúng cách.

2. Trả lời câu hỏi

1. What is Reflected Cross-Site Scripting (XSS), and how does it differ from other types of XSS attacks like Stored XSS and DOM-Based XSS?

→ Reflected Cross-Site Scripting (Reflected XSS) là một lỗ hổng bảo mật xảy ra khi dữ liệu do người dùng cung cấp được ứng dụng web phản hồi lại ngay trên trang web mà không được kiểm tra hoặc mã hóa đúng cách, cho phép kẻ tấn công chèn và thực thi mã JavaScript độc hại trên trình duyệt của nạn nhân. Khác với Reflected XSS, Stored XSS lưu trữ mã độc trên máy chủ và tự động thực thi khi người dùng truy cập nội dung bị nhiễm. Trong khi đó, DOM-Based XSS xảy ra do việc xử lý dữ liệu không an toàn bởi JavaScript phía trình duyệt, và việc thực thi mã độc diễn ra ở phía client thay vì xuất phát trực tiếp từ phản hồi của máy chủ.

2. Explain how Reflected XSS works, including how malicious scripts are injected and executed in the victim's browser, and discuss the typical attack vectors for this type of vulnerability.

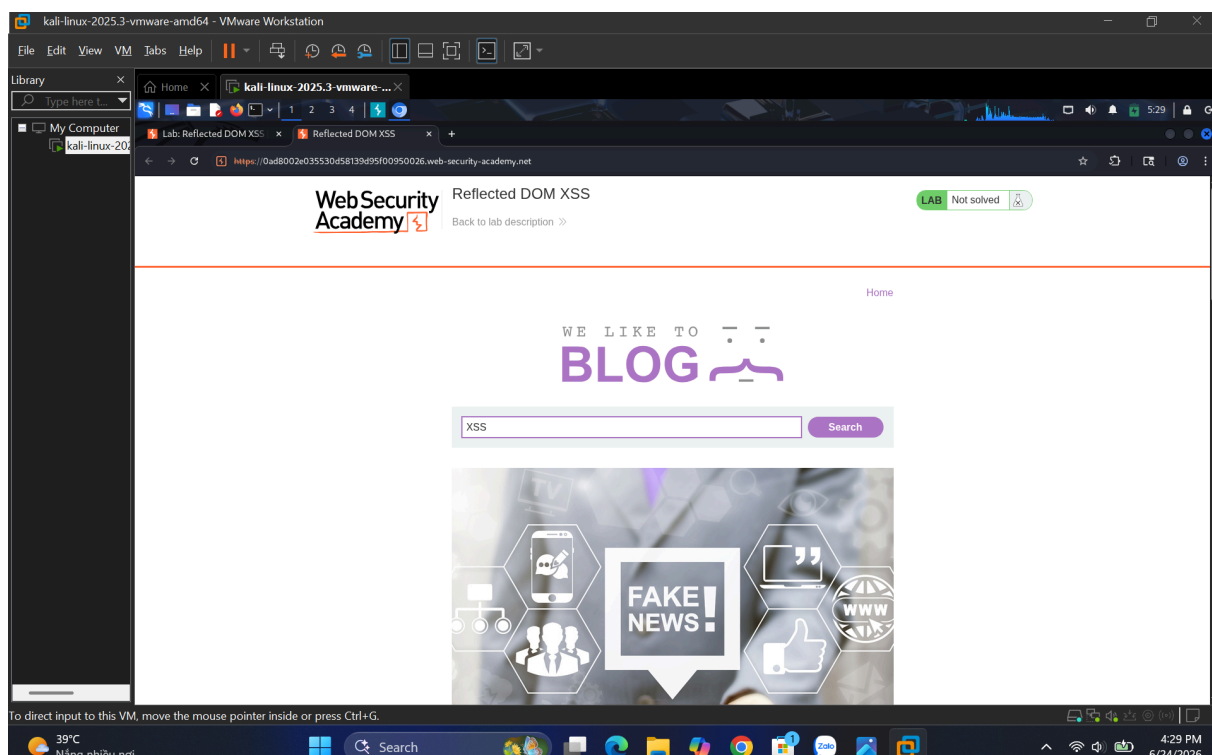
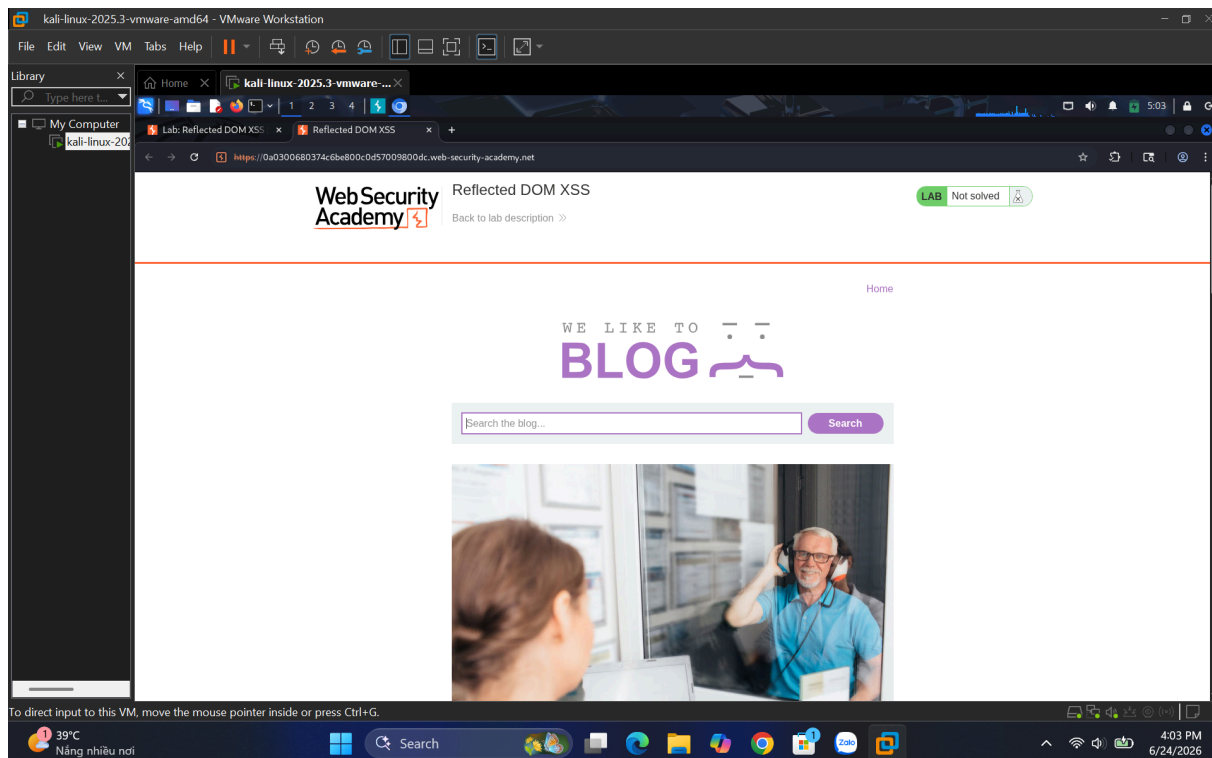
→ Reflected XSS hoạt động khi kẻ tấn công chèn mã JavaScript độc hại vào một tham số đầu vào như URL, ô tìm kiếm hoặc biểu mẫu, sau đó ứng dụng phản hồi lại dữ liệu này trên trang web mà không xử lý an toàn. Khi nạn nhân truy cập liên kết chứa payload độc hại, trình duyệt sẽ nhận phản hồi từ máy chủ và thực thi đoạn mã JavaScript đó. Điều này có thể dẫn đến việc đánh cắp cookie, chiếm quyền phiên đăng nhập hoặc thực hiện các hành động trái phép dưới danh nghĩa nạn nhân. Các vector tấn công phổ biến của Reflected XSS bao gồm tham số URL, ô tìm kiếm, biểu mẫu nhập liệu, liên kết gửi qua email, tin nhắn hoặc mạng xã hội.

3. Chi tiết bài lab

B1. Kiểm tra chức năng tìm kiếm

Truy cập website của lab và nhập chuỗi thử nghiệm vào ô search: XSS

Sau đó gửi yêu cầu tìm kiếm.

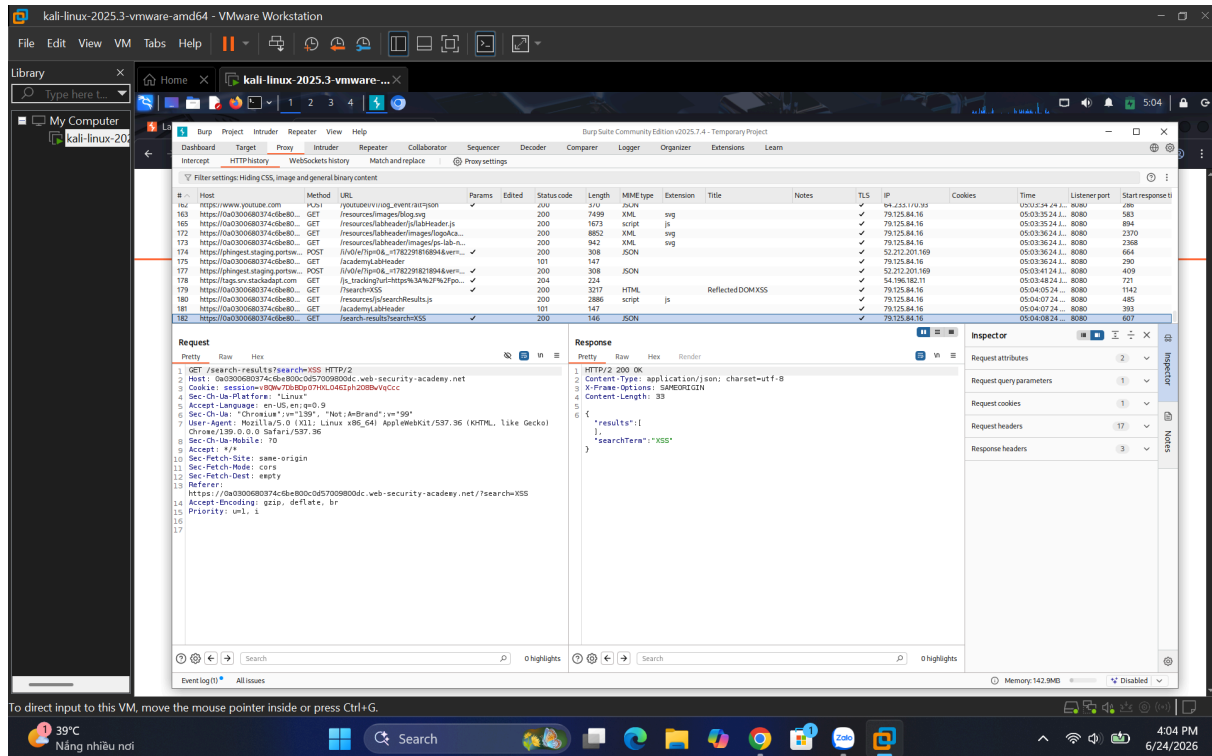


B2. Quan sát

Sử dụng Burp Suite để chặn và quan sát request/response.

Trong response, dữ liệu tìm kiếm được phản hồi dưới dạng JSON:

```
{
  "results": [
  ],
  "searchTerm": "XSS",
}
```



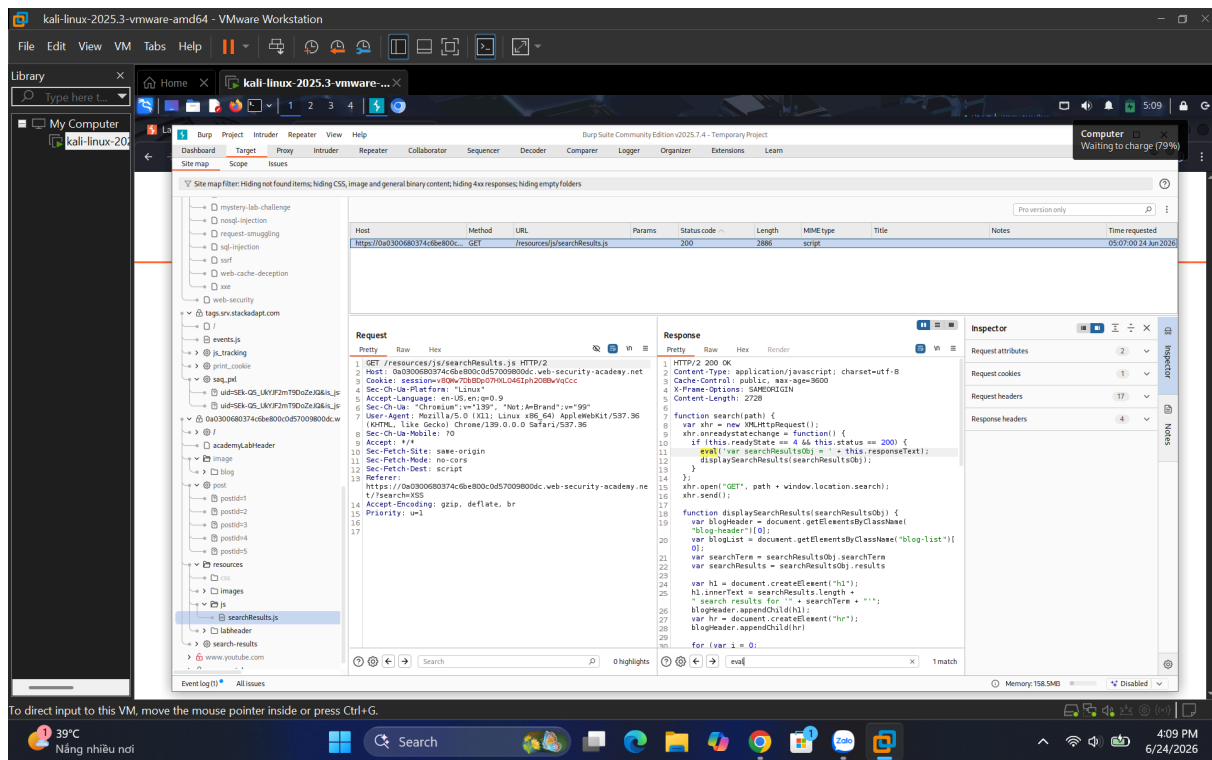
B3. Phân tích script

Mở file searchResults.js trong Site Map của Burp Suite.

Phát hiện ứng dụng sử dụng:

```
eval('(' + response + ')');
```

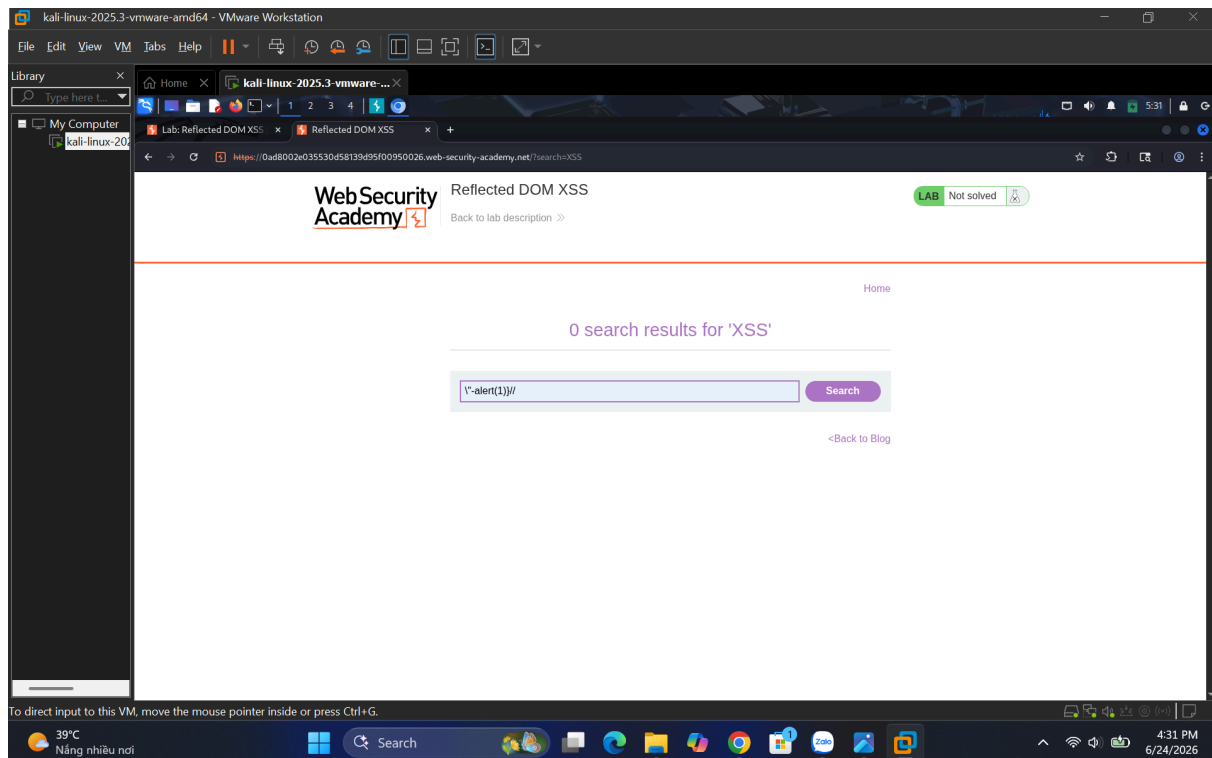
Điều này cho thấy dữ liệu JSON được thực thi bằng hàm eval(), tạo điều kiện cho việc chèn mã JavaScript độc hại.



B4: Khai thác lỗ hổng

Nhập payload vào ô search:

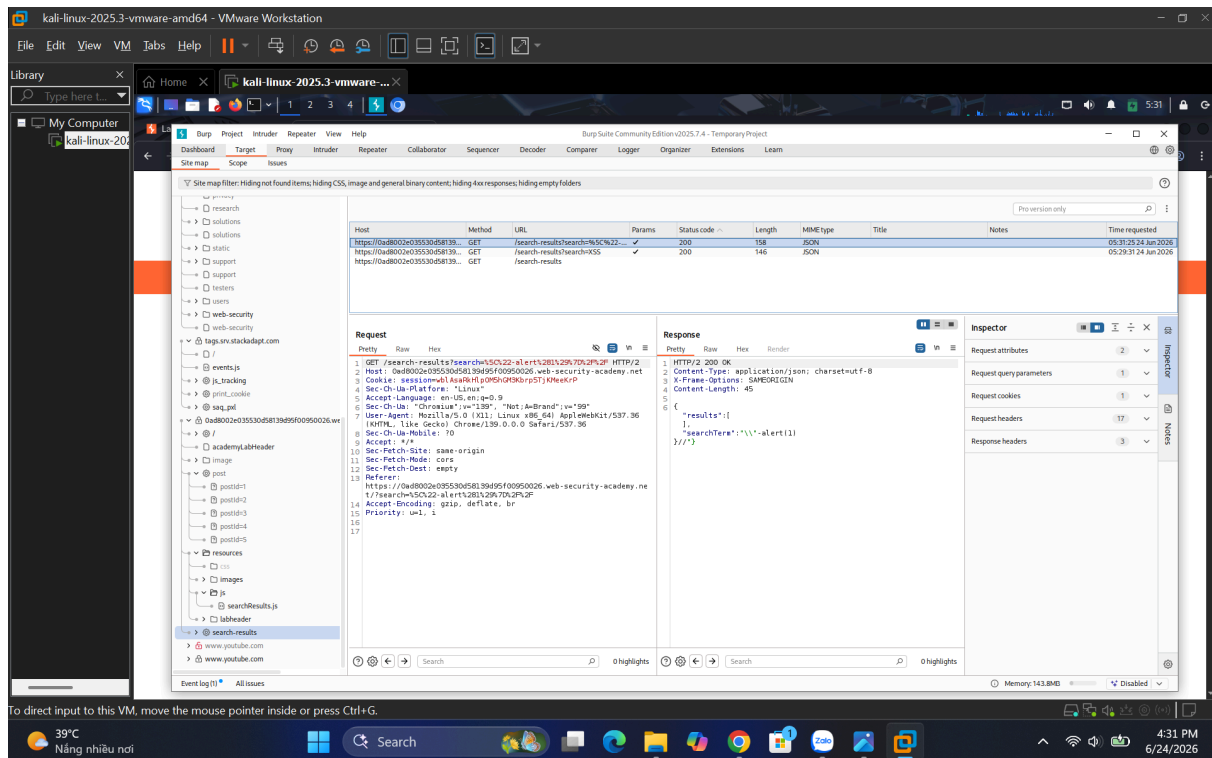
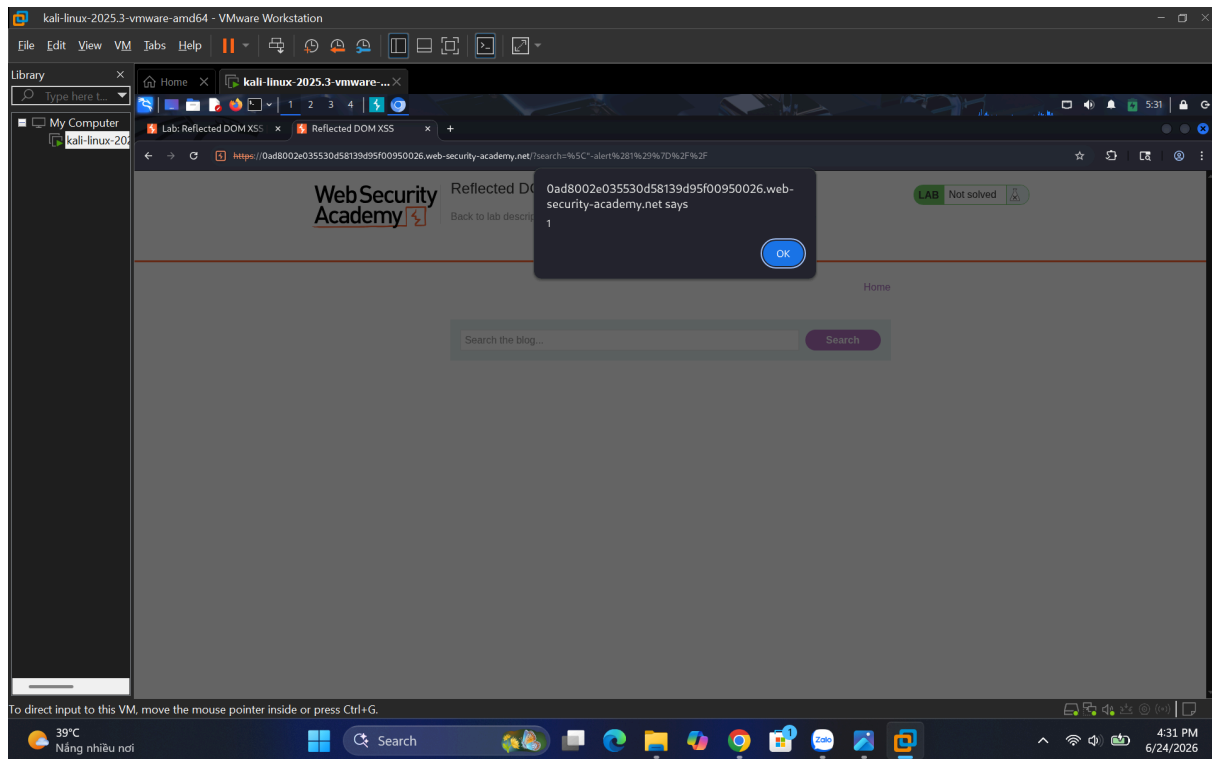
`"-alert(1)"/`



Khi hàm eval() xử lý dữ liệu trên, chuỗi bị đóng sớm và đoạn mã:

alert(1) được thực thi.

Popup xuất hiện chứng tỏ lỗ hổng XSS đã được khai thác thành công và lab được giải quyết.



Thành phần:

- \ : lợi dụng việc backslash không được escape để phá cơ chế escape dấu ngoặc kép.
- - : toán tử JavaScript dùng để tách các biểu thức.
- alert(1) : mã JavaScript được thực thi.
- } : đóng đối tượng JSON sớm.
- // : comment phần còn lại của JSON để tránh lỗi cú pháp.

B5: kết thúc và hoàn thành lab

